

Curs 7 — Geometrie Computațională: Explicație în profunzime

Acesta NU e un rezumat de slide-uri, ci o explicație de la zero: intuiția din spatele fiecărui algoritm, de ce funcționează, și exemple numerice concrete pe care le poți reface cu creionul. (Pentru varianta scurtă, orientată pe grilă, vezi sinteza separată.)

Ideea care leagă tot cursul

Geometria computațională pune o întrebare simplă: *cum scrii algoritmi eficienți pentru probleme cu puncte, segmente și poligoane?* Naivitatea costă scump — verificarea fiecărei perechi de segmente e $O(n^2)$, reconstrucția unei înfășurători „pe ghicite” e haotică. Trucul, de fiecare dată, e să **reduci problema geometrică la o problemă de structuri de date** pe care deja o știi rezolva rapid.

Și exact de-asta e capitolul ăsta în cursul de structuri de date: **Graham's scan e o aplicație a stivei**, iar **algoritmul de intersecție a segmentelor e o aplicație a arborilor red-black**. Geometria e doar haina; mecanismul dedesubt e ce ai învățat deja.

Un avertisment pe care îl face și cursul: inamicul nr. 1 în practică e **degenerarea** — trei puncte coliniare, trei drepte prin același punct. Codul real are nevoie de cazuri speciale pentru ele. În curs (și aici) le ignorăm ca să vedem ideea curată.

1. Produsul vectorial — instrumentul din care decurge totul

Aproape fiecare algoritm geometric se reduce la o singură întrebare elementară: „**virez la stânga sau la dreapta?**” Și răspunsul la asta vine dintr-un singur calcul, fără trigonometrie, fără împărțiri.

Ce este, geometric

Pentru doi vectori $p_1 = (x_1, y_1)$ și $p_2 = (x_2, y_2)$, produsul vectorial (în 2D) e un singur număr:

$$p_1 \times p_2 = x_1 y_2 - x_2 y_1$$

Geometric, acest număr e **aria cu semn a paralelogramului** construit pe cei doi vectori. Două lucruri contează:

- **Mărimea** = aria paralelogramului.
- **Semnul** = orientarea: pozitiv dacă p_2 e rotit **trigonometric (counterclockwise)** față de p_1 , negativ dacă e rotit **orar (clockwise)**.

De ce semnul codifică orientarea? Pentru că e fix determinantul matricei $\begin{bmatrix} x_1 & x_2 \\ y_1 & y_2 \end{bmatrix}$, iar determinantul îți spune dacă transformarea care duce vectorii unitate în (p_1, p_2) păstrează sau inversează orientarea planului. Pozitiv = păstrează (sens trigonometric), negativ = inversează (sens orar).

Testul de viraj (de aici vine totul)

Asta e aplicația care contează. Ai trei puncte (A, B, C) și mergi $(A \rightarrow B \rightarrow C)$. **Virezi la stânga sau la dreapta în B?** Calculezi produsul vectorial al vectorilor $(B-A)$ și $(C-A)$:

$$\text{orient}(A, B, C) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x)$$

- $> 0 \rightarrow$ **viraj la stânga** (counterclockwise);
- $< 0 \rightarrow$ **viraj la dreapta** (clockwise);
- $= 0 \rightarrow$ cele trei puncte sunt **coliniare**.

(În slide-uri, procedura **DIRECTION** folosește o permutare a argumentelor, dar principiul e identic: semnul produsului vectorial = orientarea.)

Exemple numerice (fă-le pe hârtie, e instant):

Exemplul 1 — viraj la stânga. $A=(0,0)$, $B=(4,0)$, $C=(2,2)$. $\text{orient} = (4-0)(2-0) - (0-0)(2-0) = 8 - 0 = 8 > 0$. Mergi spre est până la $(4,0)$, apoi spre stânga-sus la $(2,2) \rightarrow$ clar virezi la **stânga**. ✓

Exemplul 2 — viraj la dreapta. $A=(0,0)$, $B=(4,0)$, $C=(2,-2)$. $\text{orient} = (4)(-2) - (0)(2) = -8 < 0$. De data asta C e dedesubt $\rightarrow$ virezi la **dreapta**. ✓

Exemplul 3 — coliniare. $A=(0,0)$, $B=(2,2)$, $C=(4,4)$. $\text{orient} = (2)(4) - (2)(4) = 0$. Toate pe aceeași dreaptă. ✓

De ce e mai bun decât trigonometria

Ai putea calcula unghiuri cu atan2 și să le compari — dar asta înseamnă funcții trigonometrice, împărțiri, și erori de virgulă mobilă. Produsul vectorial e **doar înmulțiri și scăderi**: rapid, exact pe întregi, fără cazuri speciale pentru pante verticale. De-asta toți algoritmi de mai jos îl folosesc ca primitivă $O(1)$.

2. Se intersectează două segmente?

Pare banal: rezolvi sistemul de ecuații al celor două drepte, găsești punctul de intersecție, verifici dacă pică pe ambele segmente. Funcționează, dar e plin de cazuri urâte: drepte

paralele (împărțire la zero), segmente verticale (pantă infinită), coliniaritate. Soluția elegantă **nu rezolvă deloc ecuațiile** — folosește numai testul de orientare.

Etapa 1: respingerea rapidă (bounding box)

Mai întâi, un filtru ieftin. **Bounding box-ul** unui segment e cel mai mic dreptunghi cu laturi paralele cu axele care îl conține (colțurile sunt min/max ale coordonatelor capetelor). Dacă **bounding box-urile a două segmente nu se suprapun, segmentele sigur nu se ating** — și asta îl verifici instant, doar comparând coordonate min/max. E un filtru care elimină rapid majoritatea perechilor evident-disjuncte înainte să faci calcule mai serioase.

Etapa 2: testul de „încălecare” (straddle)

Acum partea isteată. Spunem că segmentul $\overline{AB}$ **încalecă** o dreaptă dacă cele două capete ale lui sunt **de o parte și de alta** a dreptei (sau pe ea). Și iată regula completă:

Două segmente se intersectează $\Leftrightarrow$ trec testul bounding box ȘI fiecare încalecă dreapta care conține celălalt segment.

Cum verifici că $\overline{CD}$ încalecă dreapta lui $\overline{AB}$? Te uiți la orientarea lui C și a lui D față de $\overline{AB}$: dacă $\text{orient}(A, B, C)$ și $\text{orient}(A, B, D)$ au **semne opuse**, atunci C și D sunt de părți diferite ale dreptei $AB \rightarrow CD$ o încalecă.

De ce ai nevoie de AMBELE încălecări

Asta e întrebarea care lămurește tot. De ce nu e destul ca un singur segment să încalece dreapta celuilalt?

Pentru că **dreptele se pot intersecta în afara segmentelor**. Imaginează-ți un segment orizontal scurt și un segment vertical mai la dreapta lui. Verticalul are capetele de o parte și de alta a dreptei orizontale (deci încalecă), dar capetele orizontalului sunt amândouă **în stânga** dreptei verticale (același parte $\rightarrow$ nu încalecă). Dreptele lor se intersectează undeva în dreapta, dar **nu pe segmentul orizontal**. Doar dacă **fiecare** segment încalecă dreapta celuilalt ești sigur că punctul de intersecție al dreptelor cade pe ambele segmente.

Exemplu care se intersectează: $\overline{AB} = (0, 0)-(4, 0)$ (orizontal), $\overline{CD} = (2, -1)-(2, 1)$ (vertical, prin mijloc).

- CD încalecă dreapta AB? $\text{orient}(A, B, C) = (4)(-1) = -4 < 0$; $\text{orient}(A, B, D) = (4)(1) = 4 > 0$. Semne opuse $\rightarrow$ da. ✓
- AB încalecă dreapta CD? $\text{orient}(C, D, A) = (0)(0-(-1)) - (2)(0-2) = 4 > 0$; $\text{orient}(C, D, B) = (0)(1) - (2)(4-2) = -4 < 0$. Semne opuse $\rightarrow$ da. ✓
- Ambele încalecă $\rightarrow$ **se intersectează** (în $(2, 0)$). ✓

Exemplu care NU se intersectează (deși dreptele se taie): $\overline{AB} = (0, 0)-(2, 0)$, $\overline{CD} = (3, -1)-(3, 1)$ (vertical, la dreapta lui AB).

- CD încalecă dreapta AB (axa x)? C dedesubt, D deasupra → da.
- AB încalecă dreapta CD (dreapta $x=3$)? $A=(0,0)$ și $B=(2,0)$ au amândouă $x < 3$ → aceeași parte → **nu**.
- Un singur segment încalecă → **nu se intersectează**. ✓ Și fix de-asta ai nevoie de ambele teste.

Cazurile coliniare

Când vreun `orient` iese `= 0`, un capăt e exact pe dreapta celuilalt segment (coliniaritate). Atunci semnul nu-ți mai spune nimic și folosești `ON-SEGMENT`, care verifică pur și simplu dacă punctul cade în intervalul de coordonate al segmentului (între min și max pe x și pe y). Astea sunt cazurile de „atingere la capăt” sau „suprapunere parțială”.

3. Se intersectează vreo pereche din n segmente? (sweep line)

Acum problema mare: ai n segmente și vrei să știi dacă **există** măcar o pereche care se taie. Naiv, verifici toate cele `$C(n, 2) \approx n^2/2$` perechi — $O(n^2)$. Pentru n mare, inacceptabil. Algoritmul cu **baleiere (sweeping)** o face în $O(n \log n)$.

Ideea baleierii

Imaginează-ți o **dreaptă verticală** care mătură planul **de la stânga la dreapta**. Coordonata x pe care se mișcă e tratată ca **timp**: „la momentul x, ce văd?”

La orice poziție, segmentele tăiate de dreapta verticală au o **ordine de sus în jos**. Cheia întregului algoritm e următoarea observație:

Două segmente se pot intersecta doar dacă, la un moment dat, devin VECINE (adiacente) în ordinea verticală.

De ce? Ca două segmente să se taie, trebuie ca în momentul de dinaintea tăierii să nu existe nimic „între” ele pe verticală — altfel ar trebui să taie mai întâi ce e între. Deci nu trebuie să verifici toate perechile, ci **doar perechile care devin vecine** pe măsură ce dreapta avansează. Asta taie costul de la $O(n^2)$ la $O(n \log n)$.

Cele două structuri de date

Algoritmul ține minte două lucruri:

1. **Sweep-line status** — ordinea de sus în jos a segmentelor tăiate *acum* de dreapta. E o **ordine totală** dinamică, peste care ai nevoie de: `INSERT`, `DELETE`, `ABOVE(s)` (vecinul de deasupra lui s), `BELOW(s)` (vecinul de dedesubt). De câte structuri ai nevoie ca să faci inserare, ștergere și „vecinul imediat mai mare/mai mic”, toate în $O(\log n)$? De un

arbore echilibrat — red-black tree. Aici se vede reducerea: problema geometrică devine o problemă de BST echilibrat.

2. **Event-point schedule** — lista coordonatelor x unde se oprește dreapta. Ordinea verticală **nu se schimbă** decât în aceste „event points”, care sunt exact **capetele segmentelor**. Le sortezi o dată, stânga→dreapta.

Algoritmul, și de ce testezi la inserare ȘI la ștergere

Parcurgi capetele în ordine. La fiecare:

- **Capăt stâng al unui segment s** (s *intră* în baleiere): îl inserezi în ordine, apoi îl testezi cu **noii lui vecini** — cel de deasupra și cel de dedesubt. (Tocmai a apărut între ei, deci s-ar putea să taie pe unul.)
- **Capăt drept al unui segment s** (s *iese*): înainte să-l ștergi, observă că cei doi foști vecini ai lui (cel de deasupra și cel de dedesubt) **devin acum adiacenți** unul cu altul → îi testezi pe **ei doi** între ei. Apoi ștergi s .

De-asta testezi în ambele momente: la inserare verifici noua adiacență creată de segmentul care intră; la ștergere verifici noua adiacență creată când un segment pleacă și lasă doi vecini față în față.

De ce e $O(n \log n)$ și de ce e corect

Costul: ai $2n$ capete (event points). Sortarea lor: $O(n \log n)$. Apoi $2n$ evenimente, fiecare făcând câteva operații pe red-black tree de câte $O(\log n) \rightarrow 2n \cdot O(\log n) = O(n \log n)$. Sortarea și parcurgerea se echilibrează la $O(n \log n)$ total.

Corectitudinea: algoritmul poate greși doar într-un fel — să *rateze* o intersecție (nu raportează una care există). Demonstrația arată că nu se întâmplă: ia prima intersecție din stânga (cea mai la stânga, iar la egalitate cea mai de jos), între segmentele a și b . Fiindcă nu există intersecții la stânga ei, ordinea din arbore e corectă peste tot la stânga. Și fiindcă presupunem că nu trec trei segmente prin același punct, există un event point — fix la stânga sau chiar la punctul de intersecție — la care **a și b devin consecutive** în ordine. La acel event point, testul **ABOVE/BELOW** le compară și prinde intersecția. Deci prima intersecție e mereu detectată.

4. Înfășurătoarea convexă (convex hull)

Ce e și de ce „convex”

Convex hull-ul unei mulțimi de puncte e cel mai mic poligon convex care le conține pe toate. Imaginea mentală perfectă: pui un **elastic** în jurul tuturor cuielor și îl lași să se strângă — forma în care se oprește e hull-ul.

„Convex” = fără adâncituri: pentru orice două puncte ale poligonului, segmentul dintre ele rămâne în interior. Punctele „dinăuntru” nu fac parte din hull; doar cele de pe margine, în colțuri, contează.

De ce e cel puțin la fel de greu ca sortarea

Un fapt frumos care explică de ce $O(n \log n)$ e bariera naturală: **dacă ai putea calcula hull-ul mai repede decât $O(n \log n)$, ai putea și sorta numere mai repede decât $O(n \log n)$** — ceea ce e imposibil în modelul cu comparații. (Iei numerele (a_i) , le mapezi la puncte (a_i, a_i^2) pe o parabolă; toate sunt pe hull, iar ordinea lor pe hull e ordinea sortată.) Deci $O(n \log n)$ e optim — și Graham's scan îl atinge.

Există însă un algoritm care poate fi *mai bun* când hull-ul are puține vârfuri: Jarvis. Vedem de ce.

5. Graham's Scan — de ce funcționează

Ideea

Trei pași: (1) alegi un punct sigur de pe hull, (2) sortezi restul după **unghiul polar** în jurul lui, (3) le parcurgi în ordine și, de fiecare dată când un punct ar crea o **adâncitură**, dai înapoi și-l scoți. „Datul înapoi” e exact ce face o **stivă** — de aceea Graham folosește o stivă.

De ce punctul cu cel mai mic y? Punctul cel mai de jos (la egalitate, cel mai din stânga), (p_0) , e garantat pe hull — nimic nu e sub el. Sortând celelalte puncte după unghiul față de (p_0) , le procesezi exact în ordinea în care le-ai întâlni mergând **trigonometric (counterclockwise)** în jurul hull-ului.

Invariantul stivei (inima corectitudinii)

În orice moment, stiva conține (de jos în sus) **fix vârfurile hull-ului punctelor procesate până acum, în ordine trigonometrică**.

Când vine un punct nou (p_i) , te uiți la ultimele două puncte din vârful stivei și faci testul de viraj spre (p_i) :

- dacă e **viraj la stânga** (counterclockwise), (p_i) continuă frumos conturul convex $\rightarrow$ îl pui pe stivă (push);
- dacă e **viraj la dreapta** (sau drept), punctul din vârf e o **adâncitură** — stă în interiorul conturului format de celelalte $\rightarrow$ îl scoți (pop). Continui să dai pop până când adăugarea lui (p_i) redevine viraj la stânga, apoi pui (p_i) .

De ce un viraj la dreapta înseamnă „scoate punctul”? Pentru că un poligon convex parcurs trigonometric face **numai** viraje la stânga. Dacă într-un colț virezi la dreapta, colțul ăla e

concav — punctul e „înfundat” spre interior și nu poate fi vârf de hull. Deci îl arunci.

Exemplu numeric complet (cu un pop)

Puncte: $((0,0), (4,0), (5,4), (2,2), (0,4))$. Cel mai de jos: $(p_0 = (0,0))$.

Unghiuri polare în jurul lui $(0,0)$: $((4,0)) \rightarrow 0^\circ$, $((5,4)) \rightarrow \approx 38.7^\circ$, $((2,2)) \rightarrow 45^\circ$, $((0,4)) \rightarrow 90^\circ$.

Ordinea sortată: $((4,0), (5,4), (2,2), (0,4))$.

- Pui pe stivă primele trei: $((0,0), (4,0), (5,4))$.
- **Procesez (2,2)**. Vârf = $(5,4)$, sub-vârf = $(4,0)$. $\text{orient}((4,0), (5,4), (2,2)) = (5-4)(2-0) - (4-0)(2-4) = 2 - (4)(-2) = 2 + 8 = 10 > 0 \rightarrow$ viraj la stânga $\rightarrow$ **push (2,2)**. Stiva: $((0,0), (4,0), (5,4), (2,2))$.
- **Procesez (0,4)**. Vârf = $(2,2)$, sub-vârf = $(5,4)$. $\text{orient}((5,4), (2,2), (0,4)) = (2-5)(4-4) - (2-4)(0-5) = 0 - (-2)(-5) = 0 - 10 = -10 < 0 \rightarrow$ viraj la dreapta $\rightarrow$ **POP (2,2)**. Stiva: $((0,0), (4,0), (5,4))$. Verific din nou: vârf = $(5,4)$, sub-vârf = $(4,0)$. $\text{orient}((4,0), (5,4), (0,4)) = (5-4)(4-0) - (4-0)(0-4) = 4 - (4)(-4) = 4 + 16 = 20 > 0 \rightarrow$ viraj la stânga $\rightarrow$ **push (0,4)**. Stiva: $((0,0), (4,0), (5,4), (0,4))$.

Hull-ul = $((0,0), (4,0), (5,4), (0,4))$. Punctul $((2,2))$ a fost scos — e în interior, nu pe margine. Exact cum trebuie.

De ce $O(n \log n)$

- Găsirea lui (p_0) : $O(n)$.
- **Sortarea după unghi polar: $O(n \log n)$** — aici e tot costul.
- Bucla principală: fiecare punct e pus o dată și scos cel mult o dată, iar testul de viraj e $O(1) \rightarrow O(n)$ în total.

Domină sortarea $\rightarrow O(n \log n)$, optim.

6. Jarvis's March — de ce funcționează

Ideea: împachetare cadou (gift wrapping)

Imaginează-ți că **înfășori o foaie de hârtie** strâns în jurul cuielor. Pleci de la punctul cel mai de jos (p_0) (același ca la Graham), tragi hârtia spre dreapta și apoi tot mai sus până atinge un punct — acela e următorul vârf al hull-ului. Continui să „înfășori” până te întorci la (p_0) .

Cum găsești următorul vârf

Din vârful curent, următorul vârf de hull e punctul față de care **toate celelalte puncte cad de aceeași parte a dreptei** — adică punctul cu **cel mai mic unghi polar** (cel „mai în sensul

orar"). Îl găsești cu o singură **trecere liniară** prin toate punctele, comparând orientări (din nou produsul vectorial). Repeți: din (p_0) alegi (p_1) , din (p_1) alegi (p_2) , ș.a.m.d. până sus la (p_k) — asta e **lanțul drept**; apoi construiești **lanțul stâng** întorcându-te, măsurând unghiuri față de axa x negativă.

De ce $O(n \cdot h)$ și când e mai bun ca Graham

La fiecare dintre cele (h) vârfuri ale hull-ului faci o trecere $O(n)$ ca să găsești următorul punct $\rightarrow O(n \cdot h)$. Important: e **output-sensitive** — costul depinde de cât de mare e **rezultatul** (numărul de vârfuri h), nu doar de input.

Compară: Graham e mereu $O(n \log n)$; Jarvis e $O(n \cdot h)$. Dacă hull-ul are **puține** vârfuri (h mic, de exemplu un triunghi în jurul a mii de puncte împrăștiate), Jarvis e $\approx O(n)$ și **bate** Graham. Dacă majoritatea punctelor sunt pe hull ($h \approx n$, de exemplu puncte pe un cerc), Jarvis devine $O(n^2)$ și **pierde** față de Graham. Alegi în funcție de cât te aștepti să fie hull-ul de „aglomerat”.

7. Imaginea de ansamblu — cum se leagă cu structurile de date

Dacă reții un singur lucru din curs, fie asta: **fiecare problemă geometrică s-a redus la o structură de date pe care o știai deja.**

- Testul de viraj (stânga/dreapta) e produsul vectorial — o primitivă $O(1)$ care înlocuiește trigonometria și apare în *absolut tot*.
- **Intersecția oricăror segmente** = baleiere + **red-black tree** (ordine dinamică cu vecini în $O(\log n)$) $\rightarrow O(n \log n)$ în loc de $O(n^2)$. Cheia: segmentele se taie doar când devin adiacente vertical.
- **Convex hull cu Graham's scan** = sortare + **stivă** (LIFO = exact mecanismul de „dat înapoi” la adâncituri) $\rightarrow O(n \log n)$, optim.
- **Convex hull cu Jarvis's march** = împachetare cadou, fără structură auxiliară, $O(n \cdot h)$ — mai bun când hull-ul e mic.

Geometria e contextul; stiva și arborele echilibrat sunt motorul. Asta e și legătura cu grila: la întrebarea „cine folosește stivă” $\rightarrow$ Graham (și DFS); la „cum se rezolvă intersecția segmentelor” $\rightarrow$ red-black tree, $O(n \log n)$, test la inserare și ștergere.

Dacă vrei, pot să-ți desenez inline (ca diagramă, nu text) cum se mișcă stiva în Graham's scan pas cu pas, sau cum baleiază dreapta verticală peste segmente — geometria se prinde mult mai bine văzând-o în mișcare. Spune-mi care algoritm vrei să-l vezi animat.